Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas
Universidad Complutense de Madrid
Óscar Martín — Curso 2026/27

24. Aplicación y composición

Aplicación no prioritaria: $

```
infixr 0 $
($) :: (a -> b) -> a -> b
f $ x = f x
```

- Es útil para evitar paréntesis anidados a la derecha.
- `(+3) (sum (map (^2) xs)) = (+3) $ sum $ map (^2) xs`

Composición: .

```
infixr 9 .
(.) :: (b -> c) -> (a -> b) -> a -> c
(f . g) x = f (g x)
```

- `(f . g . h) x = f (g (h x))`.
- Tiene prioridad alta, pero menor que la aplicación de funciones.
  - `f . g x = f . (g x)`

```
tercero = head . tail . tail
sumaCuadrados = sum . map (^2)
cuentaPares = length . filter even
```

Aplicación al revés: `&` (fuera de menú)

- No es Haskell estándar:
  - `import Data.Function ((&))`

```
infixl 1 &
(&) :: a -> (a -> b) -> b
x & f = f x
```

- `x & f & g = g (f x)`

```
tercero xs = xs & tail & tail & head
sumaCuadrados xs = xs & map (^2) & sum
cuentaPares xs = xs & filter even & length
```

Composición al revés: `>>>` (fuera de menú)

- No es Haskell estándar:
  - `import Control.Category ((>>>))`

```
infixr 1 >>>
(>>>) :: (a -> b) -> (b -> c) -> (a -> c)
(>>>) = flip (.)
```

- `(f >>> g) x = (g . f) x = g (f x)`

```
tercero = tail >>> tail >>> head
sumaCuadrados = map (^2) >>> sum
cuentaPares = filter even >>> length
```

Ejercicios

1. Evalúa *de cabeza* (sin usar el ordenador) esta expresión:
   - `map ($ 4) [(4+), (10*), (^2), (1-)]`

2. ¿Cuáles de estas expresiones son iguales a `f (g (h x))`?
   a) `(f . g . h) x`
   b) `f . g . h $ x`
   c) `f $ g $ h x`
   d) `f . g $ h x`
   e) `f . g . h x`

3. Define estas funciones sin nombrar su argumento:
   - `iniciales :: String -> String`
     - Devuelve las iniciales de cada palabra de su argumento en mayúscula.
     - `iniciales "Soy un hacha programando en Haskell"` ↝ `"SUHPEH"`.
   - `largas :: String -> [String]`
     - Devuelve una lista con las palabras con más de 5 letras de su argumento.
     - `largas "Soy un hacha programando en Haskell"` ↝ `["programando", "Haskell"]`.

4. i) Define usando `foldr` una función `componer :: [a -> a] -> a -> a` que genere la composición de todas las funciones de la lista, es decir: `componer [f, g, h] = f . g . h`.
   ii) ¿Qué tipo tiene y qué hace esta función: `(componer .) . replicate`?